

Uni-Thrift

University E-Commerce & Borrowing Platform

Fundamentals of Software Engineering

Deliverable Number: Deliverable #4 (Iteration 3 / Sprint 3)

Date of Submission: May 3, 2026

Group Members

- **Muhammad Soban Sohail** - Lead & Database Architect
- **Saad Jahangir** - Full-Stack Developer
- **Mahad Jawad Rana** - Requirement Engineer & QA

Project Title and Introduction

Project Title: Uni-Thrift: University E-Commerce & Borrowing Platform

Introduction: Uni-Thrift is a secure, campus-exclusive marketplace designed to facilitate the buying, selling, and borrowing of academic resources among students. While Sprint 2 introduced the core marketplace and transaction logic, Sprint 3 delivers the ultimate community trust and logistics features. This final iteration focuses on deploying a real-time S.O.S Borrowing Request feed, secure In-App Messaging for coordinating physical item handoffs, and a community-driven Trust Rating System to establish accountability and a thriving digital campus economy.

User Stories (Sprint Backlog)

US-3.1: SOS Borrowing Requests

Story ID	US-3.1
Story Title	SOS Borrowing Requests
Priority	Critical
Estimated Hours	10
User Story	<i>As a student in urgent need, I want to broadcast a real-time SOS request to the feed so that peers can lend me required items immediately.</i>
Description	Introduces a campus-wide emergency board. Features a real-time SOS feed, an integrated chat system trigger upon acceptance, and peer-review logic for final submission of the transaction.
Sub User Stories	As a borrower facing an upcoming exam, I want my SOS request to visually stand out as "Emergency" priority to other students in my department. As a lender, I want the system to automatically handle the temporary stock deduction of my textbook when I offer it to fulfill an SOS request.

Acceptance Criteria	The SOS feed must update dynamically with color-coded priority flags.Accepting a handshake must automatically cancel competing offers and restock their quantities.
---------------------	---

US-3.2: In-App Messaging

Story ID	US-3.2
Story Title	In-App Messaging
Priority	High
Estimated Hours	8
User Story	<i>As a buyer or seller, I want to message my transaction partner directly in the app so that we can coordinate item handoffs safely on campus.</i>
Description	A real-time, isolated chat system securely linking two users. The chat is strictly bound to a specific item transaction, ensuring communication remains organized and contextual.
Sub User Stories	As a buyer negotiating a campus meetup, I want to see the specific item details pinned at the top of our chatroom.As a privacy-conscious student, I want my chat to be completely hidden from anyone who is not explicitly the buyer or seller of that transaction.
Acceptance Criteria	Messages must be isolated using a composite check of item_id, sender_id, and receiver_id.Timestamps must accurately parse UTC server time into the user's local PC timezone.

US-3.3: Trust Rating System

Story ID	US-3.3
Story Title	Trust Rating System
Priority	Medium
Estimated Hours	6
User Story	<i>As a marketplace user, I want to rate my peers after a transaction so that the community can build a reliable, accountable trust score.</i>
Description	A 5-star rating system deployed after a successful borrow or purchase. It dynamically updates the user's global reliability score, establishing long-term accountability.
Sub User Stories	As a frequent lender, I want my profile to proudly display a dynamic 5-star rating so other students know I am reliable with returning items.As the system administrator, I want to restrict students from rating the exact same transaction multiple times to prevent rating manipulation.

Acceptance Criteria	The system must calculate the global average using a precise SQL AVG() query. Rating submissions must automatically delete the pending rating notification for that user.
----------------------------	---

Structured Specifications

US-3.1.1: SOS Real-Time Feed & Peer Review

Functional	The POST /api/sos endpoint must insert into the SOS_Requests table and trigger a system-wide Notification broadcast to all users except the requester.
Database	Upon executing sp_ResolveHandshake for an SOS, the system must execute an internal temporary table query to revert stock_quantity for any competing offers that were rejected.
UX	The UI must clearly render the priority flag (e.g., High, Emergency) with distinct visual cues on the campus Notice Board to draw immediate attention.

US-3.2.1: Messaging Isolation & Timezones

Security	The /api/messages/:itemId/:user1/:user2 route must enforce JWT token validation and ensure the authenticated user ID matches either user1 or user2 before returning the payload.
Performance	The frontend ChatPage.jsx component must utilize React's useEffect and useCallback hooks to poll for messages without causing infinite re-render loops or memory leaks.
Functional	All timestamp calculations must parse the server's GETUTCDATE() into localized strings to ensure accurate handoff coordination on campus.

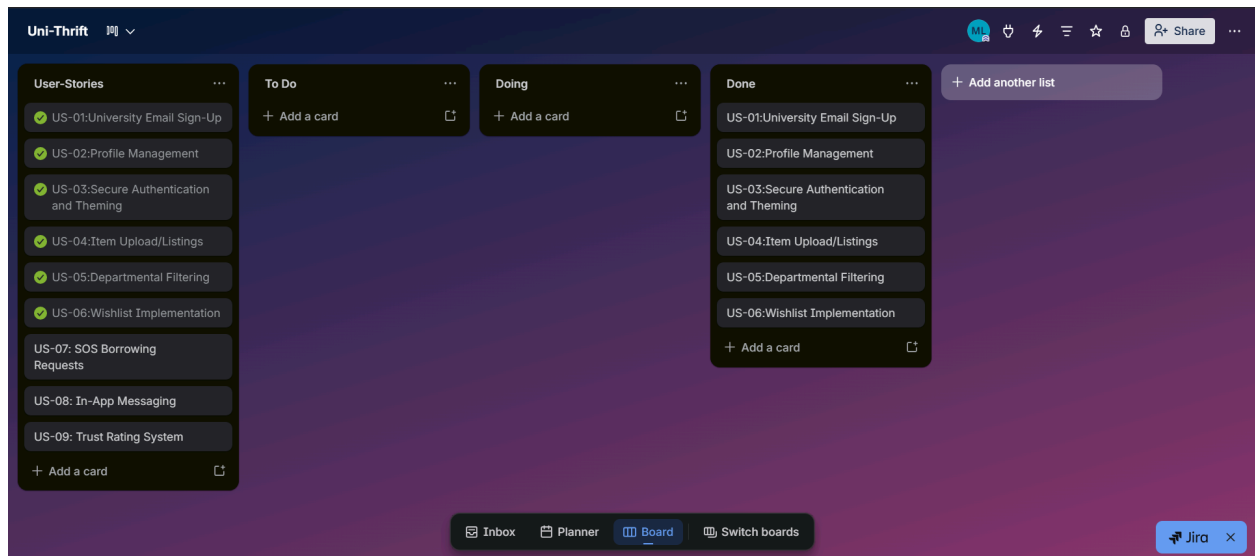
US-3.3.1: Rating Calculation & Accountability

Functional	The POST /api/ratings controller must first verify the existence of a 'completed' transaction in the Transactions table before accepting the review payload.
Database	The system updates the global reliability_score in the Users table dynamically by aggregating all historical scores using AVG(CAST(score AS FLOAT)).
UX	The ItemPage must fetch and independently render both the specific item's average rating and the seller's overall reliability score to provide maximum transparency to buyers.

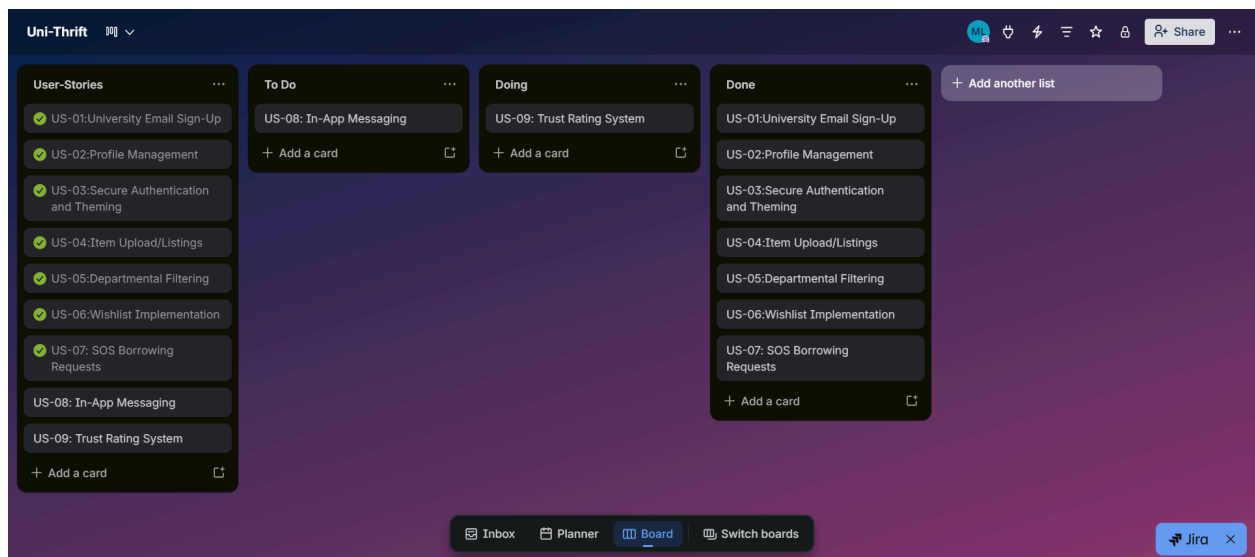
Scrum Board

<https://trello.com/b/1whRFjR1/uni-thrift>

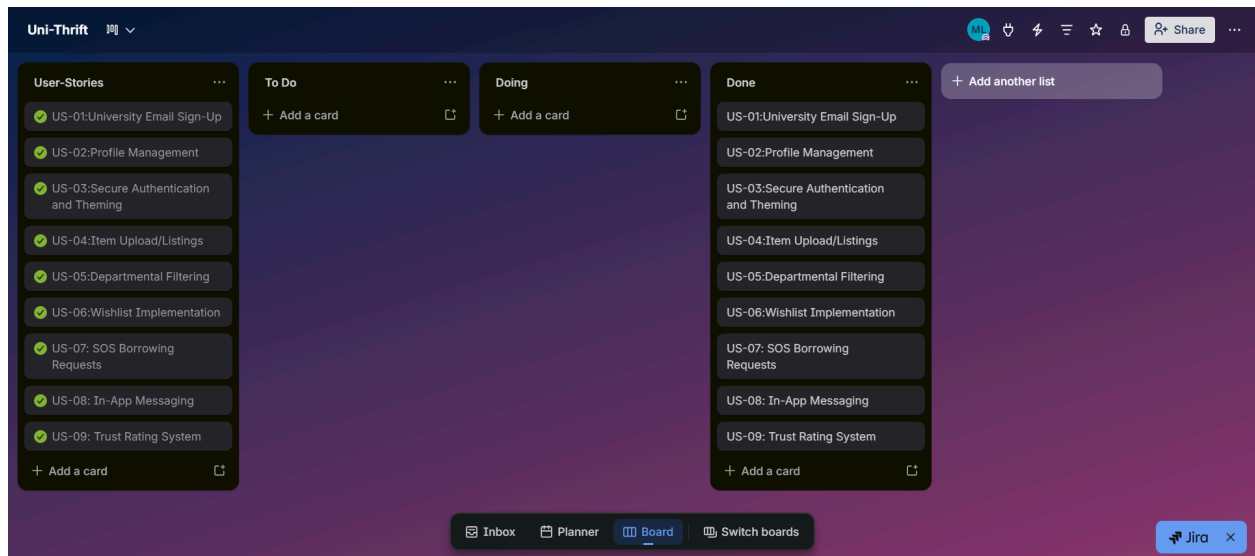
Snapshot 1 (Start of Sprint)



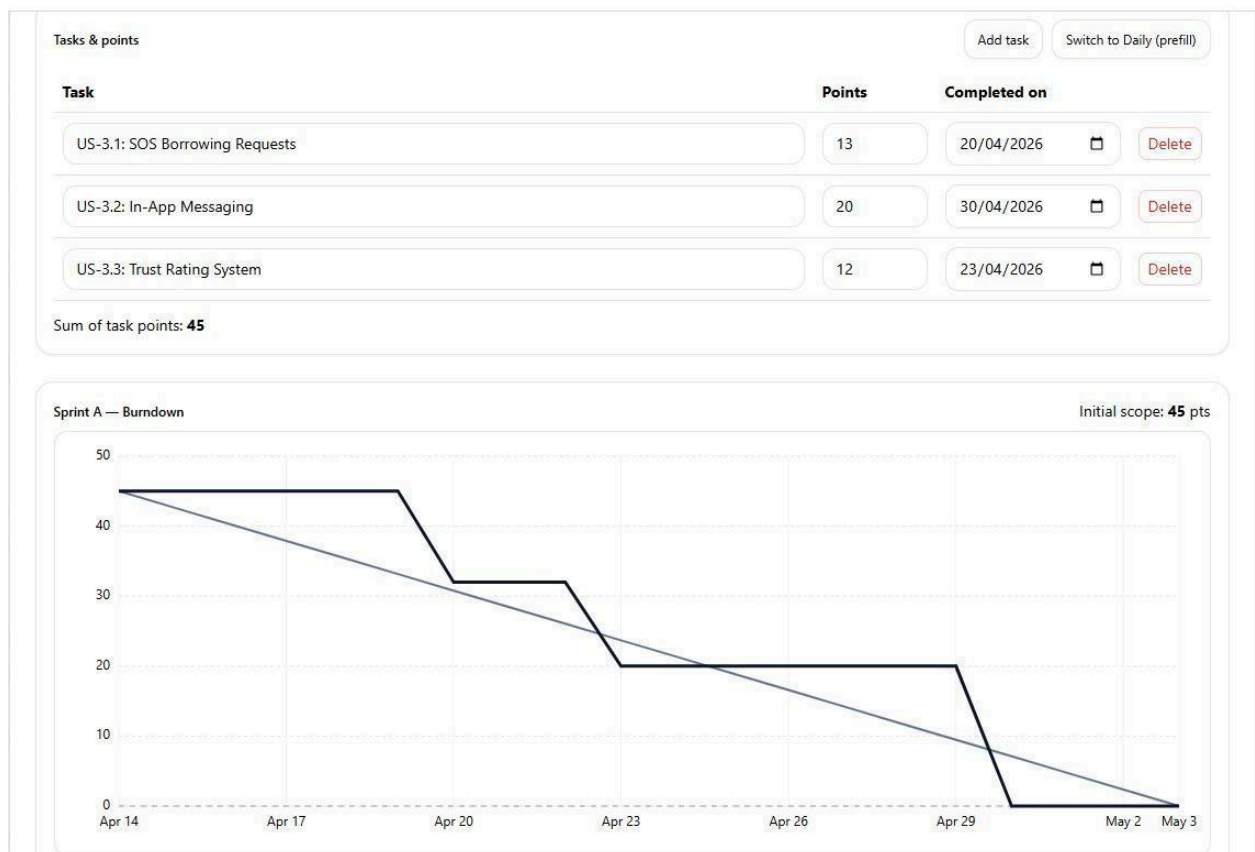
Snapshot 2 (Middle of Sprint)



Snapshot 3 (End of Sprint)



Product Burn-Down Chart



NFR Specification

Performance (Optimized Payload Handling): To accommodate students browsing the marketplace on restricted campus Wi-Fi, the backend must implement multer with strict file size limits. Furthermore, the Digital Vault must stream E-Book PDFs directly to the client via `fs.createReadStream()` rather than loading the entire file into RAM, ensuring the Node.js server maintains high throughput without memory bottlenecks.

Security (JWT & Resource Isolation): The platform strictly enforces an `authenticateToken` middleware for all transaction, wishlist, and messaging routes. In-App chat histories are hard-locked to specific `transaction_ids` through composite SQL queries, guaranteeing that sensitive physical meetup details and negotiations remain completely invisible to anyone except the explicit buyer and seller.

Reliability (UTC Timezone Normalization): Because borrow durations and physical return deadlines are strictly enforced, the MS SQL database must natively log all timestamps using `GETUTCDATE()`. This guarantees that expiration logic and chat logs function flawlessly regardless of the host machine's timezone, preventing the notorious "+5 Hours Pakistan Time" bug from prematurely revoking access to digital resources.

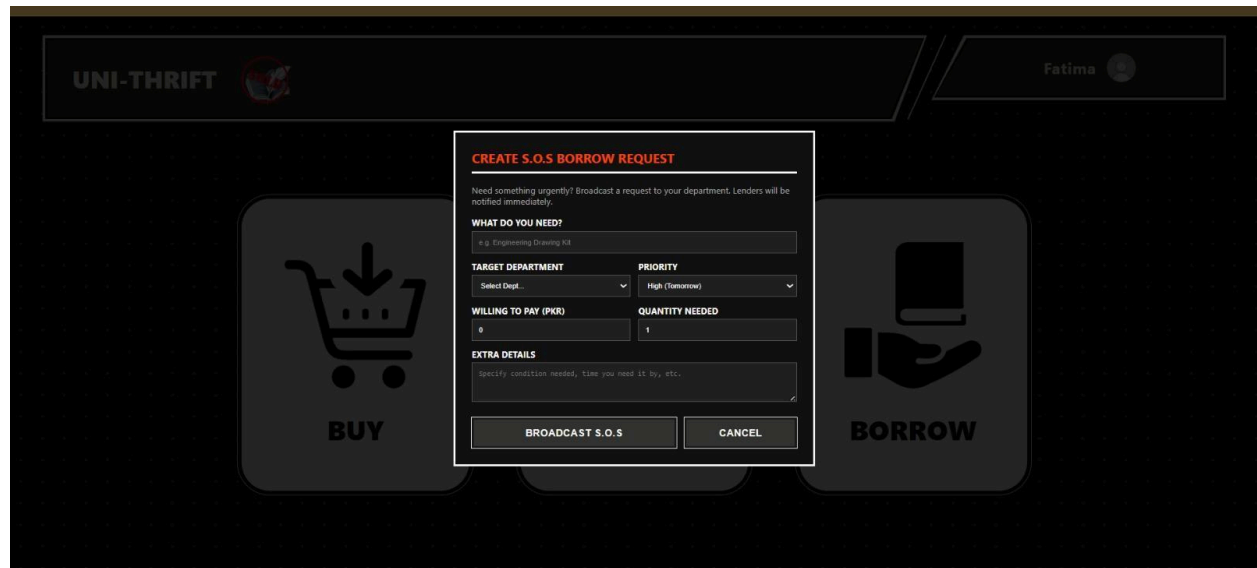
Iteration 3 Implementation Screenshots

Frontend - The Real-Time SOS Feed & Requests

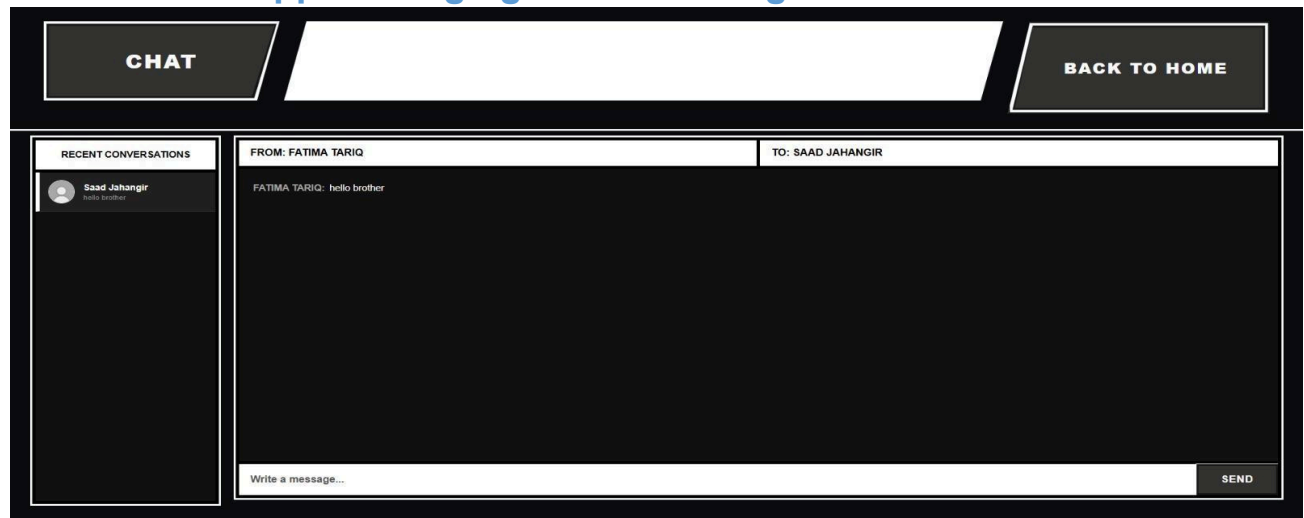
The screenshot displays a web interface for "S.O.S REQUEST" on a dark background with a grid pattern. At the top left is a button labeled "S.O.S REQUEST" and at the top right is a button labeled "BACK TO HOME". The main content area features a "CALCULATOR" form. The form includes the following fields and values:

- Requested by:** Fatima Tariq
- WILLING TO PAY:** PKR 500
- AMOUNT NEEDED:** 1
- PRIORITY:** HIGH
- DETAILS & REQUIREMENTS:** please i have exam

At the bottom of the calculator form are two buttons: "EDIT REQUEST" and "CANCEL REQUEST".



Frontend - In-App Messaging & Trust Rating Profiles



NOTIFICATION CENTER

[READ ALL DMS](#)

ACTIVE DIGITAL BORROWS



assignment 2

Expires: 5/16/2026

PENDING REVIEWS

Rate: assignment 2

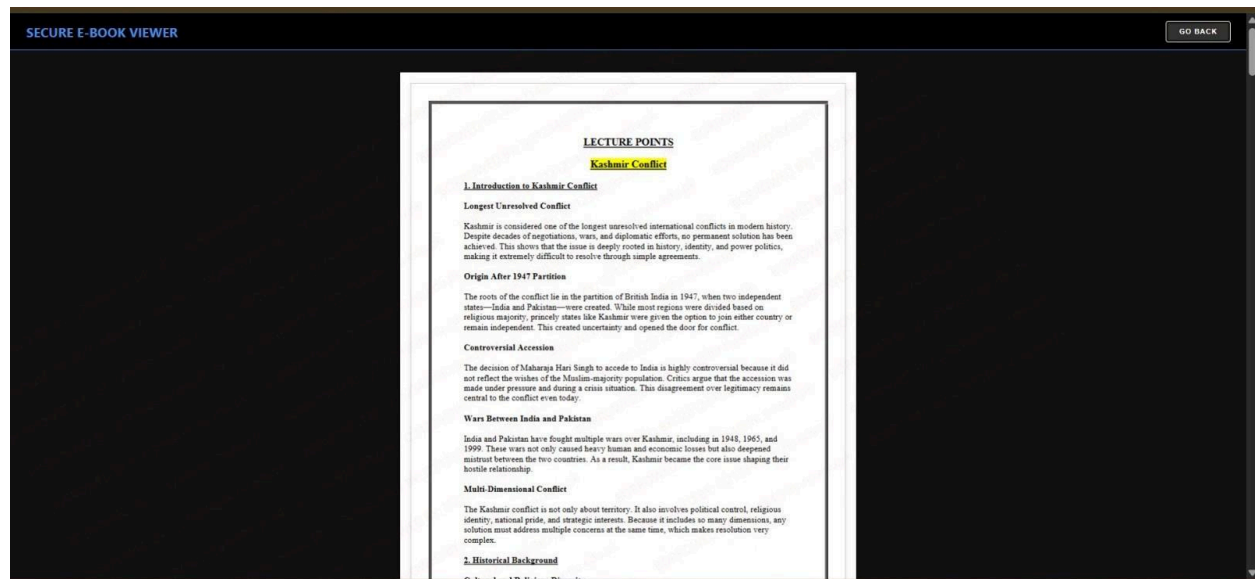
Seller: Fatima Tariq



SUBMIT RATING

CLOSE

Frontend - in browser E-Book Viewer



Work Division

Muhammad Soban Sohail (Lead & Database Architect): Designed the SQL relational schemas to support the complex relationships required for the SOS board and the Trust Rating systems. Managed transaction integrity to ensure stock quantities and escrow logic remained consistent during borrow handshakes. Oversaw overall project progress and sprint tracking on Trello.

Saad Jahangir (Full-Stack Developer): Implemented the core backend logic in Node.js, including the isolated messaging controllers and the peer-review rating algorithms. Built the responsive React.js frontend components, engineering the real-time Chatroom interfaces, the useCallback fetching loops, and dynamic UI updates for the SOS feeds.

Mahad Jawad Rana (Requirement Engineer & QA): Mapped the iteration's user stories to actionable software features and drafted strict acceptance criteria tailored to a campus environment. Conducted rigorous system and QA testing (uncovering edge-cases like timezone shifts in the messaging timestamps and ensuring the peer-review logic functioned flawlessly before final submission).